Sonar debris detector on a Raspberry Pi 4: on-device measurements

Project report, `sonar_edge`, milestone M5

10 November 2026

Scope

This report records the first measurements of the detector on the target hardware, a Raspberry Pi 4. Four questions are addressed: (i) whether the integer C runtime produces on the ARM processor exactly the output it produces on the development laptop, so that the accuracy established by cross-validation carries over unchanged; (ii) how long one frame takes, on one core and on all four; (iii) how much memory and storage the program needs; and (iv) how the cost compares with the crop classifier of the MCNN paper that motivated the project. In the course of the measurement an Advanced SIMD (NEON) kernel was added to the runtime, which made the deployed model 5.6 times faster; its design and verification are included. The data split, the network, the quantisation and the accuracy figures are described in the companion report (`report/overview.pdf`) and are summarised here only as far as needed.

Contents

1 Hardware and software **1**

2 Models under test **2**

3 What is measured and how **3**

4 Results **4**

4.1 Output equivalence on the device 4

4.2 Single-core latency, scalar code 4

4.3 NEON kernel 4

4.4 Throughput on four cores and temperature 6

4.5 Memory and program size 7

4.6 Comparison with the MCNN crop classifier 7

5 Discussion **7**

6 Limitations **8**

7 Summary **8**

1 Hardware and software

Table 1 lists the two machines. The laptop is where the runtime was developed and where all earlier timings were taken; it serves as the reference point. The Pi is a Raspberry Pi 4 Model B (revision 1.5, 4 GB), whose BCM2711 system-on-chip has four ARM Cortex-A72 cores

at 1.8 GHz, 32 KB of level-1 data cache per core and 1 MB of level-2 cache shared by the four cores. The Cortex-A72 implements the ARMv8.0 instruction set: it has the 128-bit Advanced SIMD ("NEON") vector unit, but not the 8-bit dot-product instructions (`sdot`/`udot`) introduced in ARMv8.2 and present on the Pi 5. Any vector kernel for the Pi 4 must therefore be written with 16-bit multiplies.

Table 1: Machines used. Both run the same source code; only the compiler target differs.

	Raspberry Pi 4 Model B	Laptop (reference)
Processor	Broadcom BCM2711, 4 × Cortex-A72	AMD Ryzen 5 5600H, 6 cores
Clock	1.8 GHz (governor `ondemand`)	up to 4.2 GHz
Instruction set	AArch64, ARMv8.0 + NEON	x86-64
Memory	4 GB	16 GB (14 GB usable)
Operating system	Debian 13 (Raspberry Pi OS), kernel 6.18	Arch Linux
Compiler	gcc 14.2, native	gcc 16.2, native
Cooling	no fan (none registered by the OS)	laptop fan

The runtime was copied to the Pi as source and built there with the project's makefile, i.e. `gcc -std=c11 -O2 -Wall -Wextra -Werror -pedantic`, with no other dependency than the C standard library. The `-std=c11` flag (as opposed to `gnu11`) matters for reproducibility: it disables the fusion of floating-point multiply and add into one instruction, which the ARM compiler would otherwise apply and which would change the last bits of the few floating-point operations in the box decoder.

2 Models under test

Four trained networks were measured, chosen to span the range of compute studied in the project (Table 2). All four are the fold-0 models of the cross-validation, i.e. trained on the same frames, so the test-fold accuracy column compares like with like. The pooled column is the accuracy estimate for the configuration over all five folds, averaged over seeds where more than one exists; it is the figure that describes the configuration, while the test-fold column only identifies the particular network that was run on the device.

Table 2: Models measured on the Pi. MACs: multiply-accumulate operations per frame. Fold-0 mAP: mAP@0.5 of the integer C runtime on the 439 frames of test fold 0. Pooled mAP: 5-fold cross-validation, float, mean ± s.d. over seeds (number of seeds in brackets).

Model	Input	MACs	Parameters	Fold-0 mAP	Pooled mAP
`small` (fallback)	240 × 160	34.8 M	42,463	58.9	48.1 ± 1.1 (3)
`small` (**deployed**)	360 × 240	78.4 M	42,463	63.8	52.6 ± 1.9 (3)
`small_wide`	360 × 240	203.5 M	135,359	68.1	56.4 ± 3.6 (2)
`wide`	480 × 320	450.2 M	172,351	68.6	55.2 (1)

`small` at 360 × 240 is the deployed configuration; `small` at 240 × 160 was kept as a fallback in case the deployed model proved too slow on the Pi; `small_wide` at 360 × 240 is the most accurate model found and was set aside as too expensive for the device; `wide` is the largest model trained and marks the upper end of the compute range.

3 What is measured and how

Output equivalence. The runtime is integer-only up to the final box decoding, so it should produce the same output on any processor. This is checked at three levels. (a) The *golden test* (`make test`) runs three frames through the network and compares all $45 \times 30 \times 15 = 20{,}250$ 32-bit values of the output map with values computed by the Python integer reference; a single differing bit fails it. (b) The *end-to-end test* runs the compiled program over all 439 frames of test fold 0, copies the printed detections back to the laptop and scores them with the same evaluation code and ground truth as the Python models. (c) The *byte comparison* runs the laptop build of the same program on the same 439 frames and compares the two text outputs byte for byte, including the six printed decimals of every score and box coordinate.

Latency. The time for one frame, from the 8-bit image in memory to the list of detections: the network and the decoder, i.e. the call `se_detect`. It excludes reading the file from disk, which in a deployed system is replaced by the sonar driver. It is measured by the command-line tool's benchmark mode, which calls `se_detect` repeatedly and reads the monotonic clock around each call. Ten frames spread over the test fold are each processed 50 times, giving the mean over 500 calls; this is repeated three times. The three repeats differ by less than 1% for every build except the NEON build of `wide` (3%), so a single mean is reported. The program runs on one core. The first frame of each repeat pays for cold caches; with 500 calls this contributes well under 1%.

Throughput on four cores. The runtime is single-threaded. The simplest way to use the four cores is to run four independent copies, each on its own stream of frames; this gives the throughput the board can sustain when several sonar streams, or several frames of one stream, are processed in parallel. Four copies are started together and the total number of frames divided by the elapsed time gives frames per second. The per-copy latency under this load is also recorded: it rises above the single-core value when the copies compete for the shared level-2 cache and the memory bus.

Temperature and throttling. The BCM2711 lowers its clock when the die exceeds 80 °C (soft limit) or 85 °C. The firmware reports the die temperature (`vcgencmd measure_temp`) and a bit field recording whether throttling or under-voltage has occurred since boot (`vcgencmd get_throttled`). Both were read before and after every measurement; a throttled run would have been discarded.

Memory. All buffers of the runtime are static arrays whose size is fixed at compile time: two activation buffers that alternate as layer input and output, the 32-bit output map and, for the NEON kernel, a 16-bit copy of the weights. The program allocates nothing while running. The peak resident memory of the whole process, which also counts the C library, the stack and the program code, is read from the kernel (`VmHWM` in `/proc/<pid>/status`) while the program is processing frames.

Program size. The `text` segment reported by `size`: machine code plus constant data, which includes the 8-bit weights. This is what must be stored on the device.

Energy. Not measured. The Pi 4 has no sensor for its own power draw (the Pi 5 has one), and no external meter was available. The planned method is an inline USB-C power meter or a current sensor on the 5 V supply: energy per frame is the increase of power over the idle level, divided by the frame rate.

4 Results

4.1 Output equivalence on the device

All three checks pass. The golden test passes on the Pi for all four models, with the scalar and with the NEON kernel (Section 4.3). Over the 439 test frames, the Pi printed 21,333, 25,007, 23,218 and 28,096 detection lines for the four models of Table 2, and each output file is identical byte for byte to the laptop's. The scored mAP is therefore identical too: 58.94, 63.81, 68.12 and 68.60% on the Pi and on the laptop. The NEON build's output over the full test fold is likewise identical to the scalar build's (Section 4.3).

The practical meaning is that no accuracy figure needs to be re-established on the device. The Pi runs the same function as the Python integer reference, whose accuracy was measured by cross-validation, and the quantisation itself costs nothing measurable (52.3% pooled int8 against 52.6% float for the deployed model).

4.2 Single-core latency, scalar code

Table 3 gives the latency of the original scalar runtime, the one developed and timed on the laptop. The column "GMAC/s" divides the model's multiply-accumulate count by the latency: it is the effective speed of the inner loop and makes models of different size comparable.

Table 3: Latency per frame, one core, `-O2`. Mean over 3 × 500 calls; the three repeats agree within 1%.

		Laptop, scalar		Pi 4, scalar		
Model	MACs	ms	GMAC/s	ms	GMAC/s	Pi/laptop
`small` 240 × 160	34.8 M	9.0	3.9	56.0	0.62	6.2
`small` 360 × 240	78.4 M	20.1	3.9	125.9	0.62	6.3
`small_wide` 360 × 240	203.5 M	27.6	7.4	300.1	0.68	10.9
`wide` 480 × 320	450.2 M	54.8	8.2	656.0	0.69	12.0

On the Pi the scalar runtime executes 0.62–0.69 billion multiply-accumulates per second, almost independently of the model, so latency is proportional to the MAC count: 56 ms for the fallback, 126 ms for the deployed model, 0.66 s for `wide`. The deployed model runs at 7.9 frames per second.

This is slower than the 70–100 ms projected before the measurement, which had assumed a Pi-to-laptop ratio of 3–5. The measured ratio is 6.2 for the 32-channel models and 11–12 for the 64-channel ones. Clock frequency accounts for a factor of about 2.3. The rest depends on what each compiler makes of the plain C loop. Both compilers vectorise the inner dot product at `-O2` (verified in the generated assembly), but with very different results: on the 64-channel layers the laptop reaches 7–8 GMAC/s, while the Pi stays at 0.7. The compilers also differ in version (gcc 16.2 against 14.2), and the reasons were not analysed further. What matters for deployment is that the compiler does not deliver the vector unit's throughput on the Pi, which motivated writing the vector code by hand.

4.3 NEON kernel

Design. Nearly all of the work is in the 3 × 3 convolutions. For an output pixel the runtime computes, for every output channel o,

$$\mathrm{acc}_o = b_o + \sum_{t=1}^{9} \sum_{i=1}^{c_{\mathrm{in}}} W_{o,t,i}\, x_{t,i}, \tag{1}$$

where t runs over the nine taps of the kernel, $x_{t,i} \in [0, 255]$ is an 8-bit activation and $W_{o,t,i} \in [-127, 127]$ an 8-bit weight. The scalar code evaluates this as one dot product over i per output channel and tap. The NEON kernel reorders the work so that 16 output channels are computed at once. For each tap t and input channel i, the single value $x_{t,i}$ is multiplied by the 16 weights $W_{o..o+15,\,t,\,i}$ and added to 16 running sums held in four vector registers of four 32-bit lanes each. The instruction that does this, `vmlal_laneq_s16`, multiplies four 16-bit weights by one 16-bit lane of another register and adds the four 32-bit products to an accumulator, so one instruction performs four of the multiply-accumulates of Eq. (1). Eight consecutive input values are loaded and widened from 8 to 16 bits with one instruction pair, and each of the eight lanes is then used in turn. To make the 16 weights for a given (t, i) adjacent in memory, the weights are re-laid out once at start-up from the stored order $[o][t][i]$ to $[t][i][o]$ and widened to 16 bits; this copy is a static array of $9\,c_{\text{in}}c_{\text{out}}$ 16-bit values per layer (83 KB for the deployed model). Vectorising over output channels rather than input channels has two advantages: the first layer, which has a single input channel, is vectorised as well as the others, and no horizontal sum across vector lanes is needed at the end.

Why it is exact. Every product Wx satisfies $|Wx| \le 127 \cdot 255 = 32{,}385 < 2^{15}$, and the 16-bit-by-16-bit multiply produces it exactly as a 32-bit value. The sums are accumulated in 32-bit integers as in the scalar code; integer addition is associative, so the different order of summation cannot change the result, and the bound $|\text{acc}| < 2^{28}$ established for the scalar runtime rules out overflow. The kernel is therefore bit-identical to the scalar one by construction, not merely within a tolerance. This is confirmed by the golden test on all four models and by the full test-fold output, which is identical to the scalar output byte for byte.

Selection. The kernel is compiled when the target is AArch64 and is used for every layer whose number of output channels is a multiple of 16 (all layers of all four models). Other targets, such as the laptop, compile the scalar code unchanged, and `-DSE_NO_NEON` forces the scalar path on the Pi for comparison. It adds about 80 lines to the runtime.

Results. Table 4 compares four builds of the deployed model. Letting the compiler vectorise for the specific processor (`-O3 -mcpu=cortex-a72`) gives a factor of 1.7; the hand-written kernel gives 5.6, and compiler flags add nothing further to it within the run-to-run spread. The default `-O2` build is therefore kept. Table 5 gives the gain for all four models.

Table 4: Deployed model (`small`, 360 × 240), Pi 4, one core.

Build	ms / frame	GMAC/s	vs. scalar `-O2`
scalar, `-O2`	125.9	0.62	1.0
scalar, `-O3 -mcpu=cortex-a72`	75.7	1.04	1.7
NEON, `-O2`	22.5	3.48	5.6
NEON, `-O3 -mcpu=cortex-a72`	21.5	3.65	5.9

Table 5: Pi 4, one core, `-O2`: scalar against NEON kernel.

Model	Scalar ms	NEON ms	Speed-up	NEON GMAC/s	Frames/s
`small` 240 × 160	56.0	10.2	5.5	3.41	98
`small` 360 × 240	125.9	22.5	5.6	3.48	44
`small_wide` 360 × 240	300.1	77.6	3.9	2.62	12.9
`wide` 480 × 320	656.0	140.9	4.7	3.20	7.1

The 32-channel models reach about 3.5 GMAC/s, close to what the laptop achieves with scalar code. The two models with 64-channel layers gain less (3.9 and 4.7 times, 2.6 and 3.2 GMAC/s). A plausible contributor is that the 16-bit weight table of a 64×64 layer is 74 KB (against 18 KB for 32×32), more than the 32 KB level-1 data cache, so the weights are re-read from the level-2 cache for every output pixel. This does not explain the difference between `small_wide` and `wide`, whose 64-channel layers are identical in shape, and cache misses were not measured; the cause of the lower efficiency is therefore open. Processing two or more output pixels per pass over the weights would halve the weight traffic and is the next optimisation to try for the wider models.

Figure 1 summarises the latency of all builds against the compute of the model.

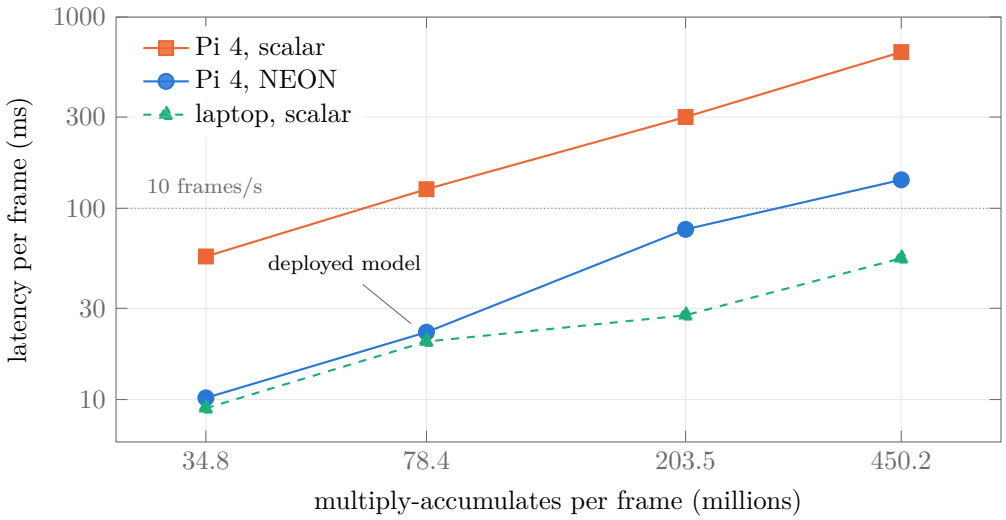


Figure 1: Latency against compute, one core, logarithmic axes. The four points of each line are the four models of Table 2. A line of slope 1 means constant GMAC/s. The dotted line marks 100 ms, i.e. 10 frames per second.

4.4 Throughput on four cores and temperature

Table 6 gives the frame rate with four copies of the program running at once, and the temperature reached.

Table 6: Four copies in parallel, one per core. "Per copy" is the mean latency of each copy under this load (range over the four copies). Temperature at the end of the run. *The firmware recorded that the soft temperature limit (80 °C) was reached during this run; see text. †Run directly after it; the firmware flag is cumulative, so whether this run was throttled cannot be told from it.

Model	Kernel	1 core, ms	4 cores, per copy, ms	Frames/s	End temp.
`small` 240 × 160	scalar	56.0	56.4–56.6	70.6	72.0 °C
	NEON	10.2	10.7–11.8	339	60.3 °C
`small` 360 × 240	scalar	125.9	126.7–126.9	31.4	68.6 °C
	NEON	22.5	25.6–26.8	149	65.7 °C
`small_wide` 360 × 240	scalar	300.1	302.6–303.8	13.2	75.9 °C
	NEON	77.6	161.5–163.8	24.4	71.5 °C
`wide` 480 × 320	scalar	656.0	663.2–664.8	6.0	81.8 °C*
	NEON	140.9	252.7–261.3	15.3	78.4 °C†

The scalar code scales perfectly to four cores: each copy is as fast as when alone. With NEON the 32-channel models lose 5–15% per copy, so four cores still give 3.3–3.5 times the single-core

rate (149 frames per second for the deployed model). The 64-channel models lose about half their speed per copy: four copies together are only 1.9 (`small_wide`) and 2.2 (`wide`) times as fast as one. A likely reason is contention for the 1 MB level-2 cache, which the four cores share and which four copies' working sets (weights and activation buffers) exceed together; as above, this is an inference from the timings.

Without a fan, every run of the three smaller models stayed at or below 76 °C and no throttling was recorded. The four-copy scalar run of `wide`, the longest run at full load (about two minutes), reached 81.8 °C, and the firmware recorded that the soft temperature limit had been reached, at which the BCM2711 begins to lower its clock. The effect on that run was small (664 against 656 ms per frame), but it shows that sustained four-core operation needs a heat sink or fan; the deployed model at four cores ran at 66 °C.

4.5 Memory and program size

Table 7: Static buffers (bss), program code and constants (text), and peak resident memory of the whole process, NEON build. The scalar build has the same text and 83–344 KB less bss (no 16-bit weight copy).

Model	Static buffers	Code + weights	Peak resident
`small` 240 × 160	429 KB	53 KB	1.8 MB
`small` 360 × 240	854 KB	53 KB	2.3 MB
`small_wide` 360 × 240	1034 KB	145 KB	2.6 MB
`wide` 480 × 320	1699 KB	181 KB	3.4 MB

The deployed model needs 2.3 MB of memory in total, of which the largest part is the two activation buffers of 355 KB each, sized for the first layer's output. The program file is 54 KB, of which 42 KB are the weights. Memory is not a constraint on any Raspberry Pi; these figures would also fit a microcontroller-class device with a few megabytes of RAM, although the latency there would be a separate question.

4.6 Comparison with the MCNN crop classifier

The MCNN of the paper that motivated this project classifies one object crop at a time. Its input is the crop zero-padded to 305×305, and it has 2.8 M parameters; counting the operations of our reimplementation gives 234.7 M multiply-accumulates per crop. The dataset has on average 2.08 annotated objects per frame (3,882 boxes in 1,868 frames, the *Wall* class included), so classifying the objects of one frame costs about 490 M multiply-accumulates. This is classification only: the crops must be found first, and the paper takes them from the manual annotations. The deployed detector finds and classifies all objects of a frame in one pass of 78.4 M multiply-accumulates, about one sixth of MCNN's classification cost, with 66 times fewer parameters (42,463 against 2.8 M). The paper's claim that MCNN is suited to embedded use was not supported by a measurement; the figures above give the comparison in operation counts, and the deployed detector's measured latency on the Pi is 22.5 ms per frame.

5 Discussion

The deployed model is fast enough. With the NEON kernel the deployed model processes a frame in 22.5 ms on one core, 44 frames per second. Forward-looking imaging sonars of the class that recorded the dataset typically deliver on the order of 10–15 frames per second, so one

core keeps up with the sensor with a factor of three or more to spare, and the other three cores remain free for the rest of the system. The 240×160 fallback is no longer needed for speed.

The accuracy reference becomes a candidate. `small_wide` at 360×240 is the most accurate configuration found (56.4 against 52.6 pooled mAP, with a seed spread that is still wide, Table 2). It had been ruled out on cost. On the Pi it now runs at 77.6 ms per frame, 12.9 frames per second on one core, which is within the sensor rate quoted above. Whether to deploy it is a trade-off between about four points of mAP and a factor of 3.4 in compute and energy; the choice depends on the power budget of the vehicle, which the energy measurement would settle.

Design choices confirmed. The decision to write the runtime in plain C with integer arithmetic and no library is what makes the equivalence result possible: the program on the Pi is verifiably the same function as the one evaluated in Python, so no accuracy is lost or needs to be re-measured in deployment. The decision to defer vector code until the target was measured is also borne out: the scalar Pi measurement showed that the compiler alone recovers only a factor of 1.7, and the hand-written kernel was then targeted at the specific instruction set of the Pi 4.

6 Limitations

- Energy per frame was not measured (no power sensor on the Pi 4, no external meter); this is the remaining item of the M5 plan.
- Thermal behaviour was observed over runs of one to three minutes on a board without a fan; the heaviest run reached the soft temperature limit. Sustained four-core operation, and operation inside a sealed housing as on an underwater vehicle, were not tested.
- Latency excludes the acquisition of the frame and its conversion to the 8-bit Cartesian image the network expects; on a vehicle this is done by the sonar's software and its cost depends on the sensor interface.
- The lower NEON efficiency of the 64-channel models, and their poor scaling to four cores, are not explained; the cache explanation offered is an inference from the timings, not a measurement of cache misses.
- Only a Pi 4 was measured. On a Pi 5 (Cortex-A76, 2.4 GHz, 8-bit dot-product instructions) a dedicated kernel using `udot` would be expected to be several times faster again; this has not been tested.
- The MCNN comparison is in operation counts, not measured time; MCNN was not ported to the runtime.

7 Summary

- On a Raspberry Pi 4 the C runtime produces output identical, byte for byte, to the laptop and bit for bit to the Python integer reference. The accuracy of the deployed model (52.6% pooled mAP, 63.8% on test fold 0) therefore holds on the device without re-measurement.
- A NEON kernel for the Pi 4's Cortex-A72, exact by construction, makes the deployed model 5.6 times faster: 22.5 ms per frame (44 frames per second) on one core, against 126 ms for the original scalar code and 76 ms for the best the compiler achieves alone.

- Four cores process 149 frames per second of the deployed model at 66 °C without a fan; only the heaviest model at full load reached the soft temperature limit. The process needs 2.3 MB of memory and a 54 KB program file.
- The most accurate model, `small_wide` at 360×240, runs at 12.9 frames per second on one core and is now a deployment candidate.
- Open: energy per frame with an external meter; a sustained thermal test; a two-pixel variant of the kernel for the 64-channel models; a Pi 5 kernel using the dot-product instructions.